

Backlog técnico de implementación

Cómo trabajar este backlog

Control

Versión 2.0 | Corte documental: 29/09/2026. Documento técnico para revisión académica. Implementado significa observado en código; no equivale a validación productiva. Las iteraciones son planes de trabajo, no actas ejecutadas.

Orden y alcance

Las diez iteraciones son paquetes de trabajo, no diez sprints ya realizados ni un calendario aprobado. I01-I05 e I08/I10 consolidan el núcleo. I06/I07/I09 son extensiones condicionadas. I09 depende de la puerta de calidad I10; sus números no obligan a publicar pagos antes de endurecer el sistema.

Scrum propuesto

Product Owner ordena valor y alcance; Developers estiman y producen incremento; Scrum Master facilita el marco. Proponer sprints de dos semanas, capacidad y fechas por acordar. Cada sprint incluye planificación, Daily Scrum, revisión y retrospectiva. Una iteración puede dividirse en varios sprints si supera capacidad.

Para un desarrollador junior

1. Leer historia y caso esperado. 2. Ubicar archivos. 3. Crear prueba que falla. 4. Cambiar una capa y validar contrato. 5. Ejecutar regresión. 6. Documentar migración y error. 7. Entregar en QA con evidencia. No copiar IDs, roles ni precios confiando en el navegador.

Definition of Ready

Historia con actor y valor, reglas, criterios positivos/negativos, dependencias, dataset y alcance aprobados. Sin un criterio comprobable, refinar antes de desarrollar.

Definition of Done

Código revisado, pruebas pertinentes aprobadas, contrato y diccionario actualizados, permisos probados, QA desplegado y evidencia vinculada a commit. Se documenta también lo pendiente. Un build exitoso no cierra aceptación ni rendimiento.

Backlog técnico de implementación

I01 | Base de datos: núcleo del marketplace

Objetivo y estado

Reproducir el esquema real y garantizar integridad en PostgreSQL. Migraciones existentes; validación de concurrencia y recuperación en PostgreSQL pendiente.

Dependencias y trazabilidad

Base de código y copias de prueba; ninguna ampliación de negocio. O2-O4; RF-01, RF-13, RF-23, RF-25; RNF-14. A09, A31.

Historias

ID	Necesidad
HU-01	Como cliente quiero que mi solicitud se conserve sin mezclarse con la de otra cuenta.
HU-02	Como técnico quiero que una oferta y su aceptación queden registradas una sola vez.

Trabajo por capa

Área	Tarea concreta
Frontend	Revisar tipos y nulabilidad consumidos en Marketplace.jsx. Mantener estados vacíos cuando no existan solicitudes. No crear tablas desde React.
Laravel	Revisar las tres migraciones de backend/api/database/migrations; probar up/down únicamente en BD desechable. Mantener FK, unique(request_id, technician_id) e índices. Corregir diferencias de database/init.sql mediante migración nueva, no editando una ya aplicada.
Python	Validar que las estructuras recibidas por /match mantengan IDs y números; no otorgar al microservicio acceso directo a la BD.
Datos	users, api_tokens, technicians, requests, offers, access_events y announcements. Diccionario físico en documento 04; tablas futuras en documento 05.

Cierre esperado

I01-01: La restricción unique rechaza la segunda fila. Desplegar BD QA aislada; ejecutar php artisan migrate --force con variables de QA y comprobar migrate:status. Nunca ejecutar migrate:fresh en la base del usuario.

I02 | Autenticación, roles y aislamiento de datos

Objetivo y estado

Asegurar que cada identidad solo opere sobre recursos autorizados. Registro, login, revocación, permisos y propiedad implementados; endurecimiento productivo pendiente.

Dependencias y trazabilidad

I01 y catálogo de permisos revisado. O3; RF-28, RF-33; RNF-08, RNF-09. A09, A12.

Historias

ID	Necesidad
HU-03	Como usuario quiero iniciar sesión y ver el panel de mi rol.
HU-04	Como administrador quiero suspender una cuenta sin permitir reutilizar sus tokens.

Trabajo por capa

Área	Tarea concreta
Frontend	Auth.jsx maneja registro e inicio; marketplace-api.js elimina sesión ante 401; Marketplace.jsx confirma actor mediante /state antes de mostrar datos. Probar cambio de cuenta con respuesta anterior demorada.
Laravel	AuthController, ApiToken y Permissions son autoridad. Registro público solo client/technician; admin se crea con marketplace:admin. profile ignora role y userId arbitrarios. Suspensión revoca tokens; no suspender administradores ni cuentas con trabajo activo.
Python	No recibe contraseñas, Bearer tokens ni direcciones para ranking. Su ruta debe estar restringida a red interna.
Datos	api_tokens guarda SHA-256 del token y expiración; users usa password hasheado e is_active. No existe tenant_id: aislamiento actual por propiedad. Multitenencia futura requiere diseño y migración específicos.

Cierre esperado

I02-01: 403 y ningún cambio de estado. Migrar acceso en QA; crear administrador por comando interactivo sin contraseña predeterminada; smoke de tres roles. Tokens y secretos fuera de Git.

Backlog técnico de implementación

I03 | Habilitación y condiciones del servicio

Objetivo y estado

Controlar elegibilidad y mostrar condiciones económicas antes de confirmar. Habilitación, disponibilidad, tarifa y cobertura implementadas. Cobros y suscripción no existen.

Dependencias y trazabilidad

I01-I02; definir qué evidencia revisa realmente el administrador. O2-O3; RF-07, RF-18, RF-30. A07, A09, A13.

Historias

ID	Necesidad
HU-05	Como técnico quiero gestionar tarifa y cobertura sin cambiar acuerdos activos.
HU-06	Como cliente quiero confirmar la tarifa de visita del técnico propuesto.

Trabajo por capa

Área	Tarea concreta
Frontend	TechnicianProfile.jsx permite tarifa y zonas; bloquea edición con ofertas pendientes o atenciones. Admin muestra verificación y AccessManager suspensión. Mostrar que visita no incluye necesariamente materiales ni reparación completa.
Laravel	technician-profile obtiene identidad de sesión, incluye zona principal y valida fee 0..100000. Bloquear cambio con ofertas pendientes o estados proposed/confirmed/in_progress. verify solo admin; nuevas cuentas técnicas no verificadas ni disponibles.
Python	Elegibilidad exige verified y available; no aceptar un pago como prueba de competencia. Probar exclusiones antes del ranking.
Datos	technicians.fee es referencia actual; requests.agreed_fee se copia al aceptar. La revisión documental detallada no tiene entidad propia: proponer verification_reviews antes de afirmar certificación.

Cierre esperado

I03-01: No se modifica verificación. Publicar incremento en QA; administrador habilita una cuenta sintética; técnico activa disponibilidad y cliente revisa condiciones.

Backlog técnico de implementación

I04 | Matching, disponibilidad y ciclo de solicitudes

Objetivo y estado

Asignar y reasignar sin duplicados, respetando plazos y confirmación. Algoritmo y ciclo implementados; integración real, concurrencia y calibración local pendientes.

Dependencias y trazabilidad

I01-I03; fixture y reglas versionados. O2-O4; RF-06 a RF-19, RF-23. A03, A05, A06, A15.

Historias

ID	Necesidad
HU-07	Como cliente quiero recibir una propuesta de un técnico elegible.
HU-08	Como técnico quiero responder antes del vencimiento sin doble asignación.

Trabajo por capa

Área	Tarea concreta
Frontend	Mostrar estado y plazo real en RequestCard; no convertir contador en autoridad. Deshabilitar envío mientras procesa y mostrar 409 si otro actor ganó. Explicar ausencia de cobertura.
Laravel	Marketplace::dispatch, tick y action conservan ventana global. Procesador cada segundo mediante schedule:work. Mantener bloqueo de solicitud y técnico. Evaluar idempotencia de create antes de reintento automático; hoy no se observa clave idempotente.
Python	POST /match ordena elegibles y devuelve rulesVersion. Laravel contrasta IDs y puntajes con fallback local; timeout total 2 s, conexión 1 s. Probar caída, JSON inválido y versión divergente.
Datos	Ofertas pending/accepted/declined/expired/cancelled; no reiniciar pending_until al reasignar. Agreed_fee fija condición. Dirección solo asignado después de confirmación.

Cierre esperado

I04-01: Como máximo tres ofertas pendientes. Desplegar API y Python juntos en QA; ejecutar worker de plazos supervisado y reiniciarlo durante una solicitud de prueba. Verificar recuperación sin reiniciar ventanas.

Backlog técnico de implementación

I05 | Frontend de técnico, administrador y UX

Objetivo y estado

Hacer operables las tareas de proveedor y supervisión con estados claros. Paneles y anuncios por audiencia implementados; auditoría móvil y accesibilidad integral pendiente.

Dependencias y trazabilidad

I02-I04 y contratos estables. O1-O3; RF-25, RF-30, RF-31; RNF-03, RNF-04. A01, A12, A27.

Historias

ID	Necesidad
HU-09	Como técnico quiero distinguir ofertas, trabajos activos e historial.
HU-10	Como administrador quiero verificar perfiles y revisar cobertura por rubro.

Trabajo por capa

Área	Tarea concreta
Frontend	Marketplace.jsx, AccountPanel.jsx, TechnicianProfile.jsx y Announcements.jsx. Revisar filtros, vacíos, busy, errores, teclado y foco visible. Mantener landing separada del panel y demo rotulada. No añadir roles elegibles en login real.
Laravel	state filtra solicitudes por propietario/ofertas; users y accessEvents solo admin. announcement-save valida audiencia, texto, URL e imagen; borrar/archivar no equivale a perder historial de servicios.
Python	No agregar endpoints de interfaz al motor de matching. Verificar que fallos de ranking no dejen pantalla sin mensaje.
Datos	Anuncios draft/published/archived para client/technician/both; catálogo de acceso visible sin hashes ni tokens.

Cierre esperado

I05-01: Respuesta antigua no reemplaza actor actual. Compilar Vite y publicar QA; smoke técnico y admin con identidades sintéticas. Registrar capturas de escritorio y móvil y defectos de accesibilidad.

Backlog técnico de implementación

I06 | Asistente determinista y registro guiado

Objetivo y estado

Proponer captura conversacional de una falla sin IA generativa. Ampliación propuesta, no implementada. Requiere aprobación del Product Owner antes de desarrollo.

Dependencias y trazabilidad

I02-I04; validar que el formulario actual resulta insuficiente. O1; extensión EX-CHAT-01, relacionada con RF-01/02. A23, A25, A31.

Historias

ID	Necesidad
HU-11	Como cliente quiero describir mi necesidad paso a paso y revisar el resumen antes de enviarlo.

Trabajo por capa

Área	Tarea concreta
Frontend	Crear componente propuesto ServiceAssistant con pasos rubro -> zona -> dirección -> descripción -> resumen. Atrás no pierde campos. Mantener formulario alternativo. Las respuestas son plantillas; no diagnósticos técnicos.
Laravel	Reusar validación de create; añadir borrador persistente solo si se aprueba. Crear la solicitud únicamente al confirmar y con idempotencia del envío. No abrir chat libre entre usuarios en esta iteración.
Python	Sin LLM y sin nuevo servicio obligatorio; la clasificación manual por catálogo es suficiente. El ranking sigue por /match al confirmar.
Datos	Propuesta service_drafts: owner, paso, datos y expiración. No almacenar conversación completa por defecto. Definir retención y borrado antes de activar.

Cierre esperado

I06-01: Resumen cambia; no hay solicitud prematura. Bandera de función apagada por defecto; habilitar en QA para usuarios de prueba. Rollback apaga bandera sin borrar solicitudes ya creadas.

Backlog técnico de implementación

I07 | LLM opcional: parseo y controles

Objetivo y estado

Evaluar extracción de campos desde texto manteniendo decisiones deterministas. Experimento propuesto, no implementado ni aprobado para producción.

Dependencias y trazabilidad

I06; dataset etiquetado, proveedor, costo y tratamiento de datos aprobados. O1-O3; EX-IA-01. A22 como riesgo de referencias inventadas; A21 solo apoyo al desarrollo.

Historias

ID	Necesidad
HU-12	Como cliente quiero una sugerencia de rubro que pueda revisar y corregir antes de enviar.

Trabajo por capa

Área	Tarea concreta
Frontend	Mostrar campos sugeridos y corrección manual. Pedir zona/dirección por controles explícitos; no inferir identidad. Si falla el parser, volver al flujo determinista sin perder borrador.
Laravel	Proponer adaptador server-side que valida JSON cerrado y lista permitida de especialidades. No entregar credenciales al navegador. No permitir tool calls, SQL, confirmación, tarifas o asignación por salida del modelo.
Python	Si se aloja parser en FastAPI, separar /parse propuesto de /match existente. Entrada limitada; salida {specialty, description, needsClarification}; validar enum y tamaño. Regla incierta -> null/pregunta, nunca inventar técnico.
Datos	Guardar solo resultado aceptado y versión del parser si se aprueba; registro de evaluación anonimizado, sin direcciones ni tokens enviados al proveedor por defecto.

Cierre esperado

I07-01: Sin cambio de permisos ni acción. Solo QA, bandera apagada, cupo de costo y timeout fijados antes del ensayo. Comparar con baseline sin LLM. Activar únicamente si existe mejora medida sin pérdida de seguridad.

Backlog técnico de implementación

I08 | Frontend cliente y flujo completo

Objetivo y estado

Validar de extremo a extremo la experiencia de solicitud a calificación. Formulario y ciclo básico implementados; asistente opcional y E2E integral pendientes.

Dependencias y trazabilidad

I02-I05; I06 solo si se habilita asistente. I07 no es obligatorio. O1-O4; RF-01, RF-18, RF-22, RF-23, RF-25. A01, A14, A31.

Historias

ID	Necesidad
HU-13	Como cliente quiero revisar tarifa, confirmar, seguir y calificar mi atención con claridad.

Trabajo por capa

Área	Tarea concreta
Frontend	RequestForm, RequestCard y Auth: mantener sesión, errores de red y datos ingresados; confirmar tarifa de visita y estado sin falsa promesa de llegada. Revisar video con movimiento reducido y acceso directo al formulario.
Laravel	Verificar create, confirm, cancel y rate con autorización; calificar una vez, entero 1..5 y antes de 48 h. El historial no debe atribuir pagos inexistentes ni reporte técnico que todavía no se captura.
Python	Probar flujo con Python activo y caído sin cambiar resultado esperado del ranking.
Datos	Una calificación por solicitud en requests; promedio y reviews del técnico se actualizan en transacción. El estado completed permanece si no se califica: resolver discrepancia RF-24 antes de afirmar cierre automático.

Cierre esperado

I08-01: Estados correctos, eventos y promedio consistente. Smoke completo en QA con cliente, técnico y admin distintos. Conservar IDs sintéticos y capturas sin datos personales. Publicar evidencia de camino feliz y excepciones.

I09 | Pagos y operación productiva: extensión condicionada

Objetivo y estado

Diseñar cobro verificable sin alterar el núcleo hasta aprobar el modelo comercial. No implementado; diseño propuesto. No hay suscripción ni pasarela configurada.

Dependencias y trazabilidad

Decidir si se cobra visita, comisión o suscripción; proveedor y obligaciones operativas; I02/I08/I10. RF-32 previo (método de pago), EX-PAY-01 (cobro real nuevo). A10, A15, A32.

Historias

ID	Necesidad
HU-14	Como cliente quiero conocer qué pago y recibir un estado verificable sin cargos duplicados.

Trabajo por capa

Área	Tarea concreta
Frontend	Proponer checkout alojado por proveedor; mostrar pendiente, pagado y fallido desde servidor. Redirección exitosa no prueba pago. No pedir tarjeta en formularios propios.
Laravel	Proponer pagos, intentos y eventos únicos por proveedor. Verificar firma de webhook, monto, moneda y recurso; transacción idempotente. Conciliación y reembolso con auditoría. No activar una suscripción inexistente al volver de checkout.
Python	Matching no participa en cobros ni recibe datos financieros. No cambiar ranking para privilegiar pagos sin regla de producto explícita.
Datos	payment_intents, payment_events y refunds propuestos. Importe en unidades menores enteras, moneda, referencia única, estado; sin PAN/CVV. Tabla de suscripciones solo tras aprobación independiente.

Cierre esperado

I09-01: No duplicar cargo ni degradar pago confirmado. Sandbox de proveedor primero; documentación y secretos separados. Producción requiere decisión comercial, QA de seguridad y responsable de conciliación. No se realizan pagos reales en esta tarea.

Backlog técnico de implementación

I10 | Seguridad, calidad y despliegue previo al lanzamiento

Objetivo y estado

Cerrar riesgos del núcleo con evidencia y autorizar una versión candidata. Pruebas unitarias/HTTP parciales existentes; carga, PostgreSQL real, restauración y auditoría integral pendientes.

Dependencias y trazabilidad

I01-I05/I08 para núcleo. I06/I07/I09 únicamente si entran en la versión. O3-O5; RF-33; RNF-01 a RNF-14. A12, A18, A29.

Historias

ID	Necesidad
HU-15	Como responsable de operación quiero desplegar y revertir una versión sin perder datos ni exponer cuentas.

Trabajo por capa

Área	Tarea concreta
Frontend	Auditar teclado, reflujo móvil, errores, contraste y movimiento reducido; comprobar que videos no bloqueen login. Revisar dependencias, tamaño y carga de secuencias.
Laravel	Pruebas negativas, límites, contraseñas y tokens; restringir debug y datos en logs. Ensayar aceptación concurrente y tareas de plazos en PostgreSQL. Revisar acceso al matching, uploads de anuncios y recuperación.
Python	Pruebas de contrato, payloads límite, versión de reglas y caída. Restringir red y medir consumo; no exponer /match directamente a Internet.
Datos	Respaldo cifrado y restauración ensayada con objetivos acordados; política de retención y eliminación pendiente de revisión. Datos sintéticos para pruebas de carga.

Cierre esperado

I10-01: Funciona con API, BD y worker reales. Puerta de salida: build, pruebas, migraciones, smoke, respaldo restaurado, seguridad y aceptación. Registrar imagen/commit, entorno, aprobador y reversión. Producción requiere aprobación final del responsable.